

Détection de deepfakes vocaux : validation et extension du modèle AASIST

Aristide Bailly

Projet Intégrateur 2A, Télécom Paris, 2025–2026

Référence principale : Jung et al., *AASIST : Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks*, ICASSP 2022.

Résumé. La synthèse vocale neuronale permet aujourd’hui de produire des voix artificielles difficiles à distinguer d’une voix réelle, ce qui menace directement les systèmes de vérification du locuteur. Ce projet part du modèle AASIST, un modèle de détection de deepfakes vocaux qui combine une convolution sinusoïdale (SincConv) et deux réseaux d’attention sur graphes (spectral et temporel), pour en valider les performances publiées puis en explorer les limites sur le dataset ASVspoof 2019 LA. J’ai d’abord validé les poids officiels (EER 0,83 %, min-tDCF 0,0275, conforme au papier à la quatrième décimale), puis reproduit l’entraînement des quatre architectures de référence (AASIST, AASIST-L, RawGAT-ST, RawNet2), qui retrouvent ou approchent les résultats publiés. L’analyse par type d’attaque montre des profils d’erreur complémentaires entre modèles, en particulier sur les attaques A17/A18 (vocoders neuronaux), ce qui m’a conduit à fusionner les scores de façon pondérée : la fusion atteint 0,73 % d’EER, sous le meilleur modèle seul. J’ai ensuite testé trois variantes du protocole d’entraînement (frequency augmentation, noise augmentation, fine-tuning), chacune comparée à AASIST from scratch (1,335 %) plutôt qu’aux poids officiels : la frequency augmentation améliore l’EER global d’AASIST (−0,19 pt) mais dégrade légèrement celui d’AASIST-L (+0,11 pt) tout en améliorant sa robustesse sur A18, et seule la noise augmentation dégrade clairement la performance (+0,28 pt). Une expérience de distillation de connaissance (AASIST → AASIST-L, $\alpha = 0,5$, $T = 4$) a produit un EER de 1,39 %, supérieur de 0,40 pt à l’entraînement from scratch d’AASIST-L (0,99 %), ce qui suggère que la distillation n’apporte pas de gain sur ce dataset. La mesure de robustesse au bruit gaussien révèle une dégradation brutale dès 20 dB SNR (EER 4,20 %) et un effondrement quasi-total à 0 dB (63,10 %), cohérent avec l’absence totale de bruit dans ASVspoof 2019 LA.

Mots-clés : anti-spoofing audio, deepfake vocal, ASVspoof 2019, graph attention network, fusion de systèmes, distillation de connaissance.

1 Introduction

La prolifération des techniques de synthèse vocale neuronale pose un problème de sécurité de plus en plus important, car il est aujourd’hui possible de générer des voix artificielles convaincantes, utilisables pour contourner des systèmes de vérification du locuteur (ASV, *Automatic Speaker Verification*). Ces attaques, appelées *spoofing* ou *deepfakes vocaux*, se divisent en deux grandes catégories : la synthèse vocale (TTS, *Text-To-Speech*) et la conversion vocale (VC, *Voice Conversion*).

Je me suis basé sur le défi ASVspoof, en particulier son édition 2019 qui reste la référence la plus utilisée dans la littérature. Organisé depuis 2015, il standardise l’évaluation des systèmes de contre-mesure (CM, *CounterMeasure*) capables de distinguer une voix authentique (*bona fide*) d’une voix falsifiée (*spoof*).

Publié à l’ICASSP 2022, AASIST (Jung et al.) s’imposait alors comme le meilleur système de détection sur ASVspoof 2019 LA, mais le papier original se concentre sur la performance brute du modèle final et laisse plusieurs questions ouvertes : comment se comporte-t-il en conditions dégradées (bruit) qui n’apparaissent pas dans ASVspoof 2019 LA ? Ses erreurs sont-elles vraiment différentes de celles de modèles plus anciens comme RawNet2, au point qu’une fusion de scores apporte un gain réel ? Et les techniques d’augmentation habituelles sont-elles seulement utiles sur un dataset déjà propre et standardisé comme celui-ci ? Ce sont ces questions, plutôt que la seule performance du modèle, qui structurent ce projet.

Lors de ce projet j’ai :

- validé l’implémentation officielle d’AASIST en réévaluant les poids pré-entraînés fournis par les auteurs ;

- reproduit l’entraînement des quatre architectures de la même famille de contre-mesures (AASIST, AASIST-L, RawGAT-ST, RawNet2) depuis une initialisation aléatoire ;
- analysé les erreurs modèle par modèle et attaque par attaque, pour comprendre où et pourquoi chacun échoue ;
- évalué une fusion pondérée des quatre systèmes ;
- testé trois variantes du protocole d’entraînement (augmentation fréquentielle, bruit, fine-tuning), chacune isolée pour mesurer sa contribution propre ;
- testé une distillation de connaissance (AASIST $\rightarrow$ AASIST-L, résultat : 1,39 % EER, supérieur de 0,40 pt au from scratch) ;
- mesuré la robustesse au bruit gaussien à cinq niveaux de SNR (0 à 30 dB), montrant un effondrement à 63 % EER à 0 dB.

2 Méthode

2.1 Architecture AASIST

AASIST traite un signal audio brut de 4 secondes (64 600 échantillons à 16 kHz) et produit un score continu : plus il est élevé, plus le signal est probablement un deepfake. Le pipeline est entièrement différentiable et se décompose en six blocs enchaînés, détaillés ci-dessous avec les dimensions de sortie (bs = taille de batch).

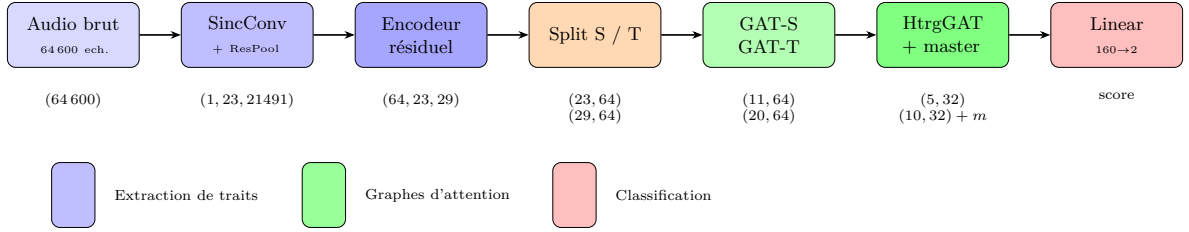


FIGURE 1 – Vue schématique du pipeline AASIST avec les dimensions de tenseurs à chaque étape.

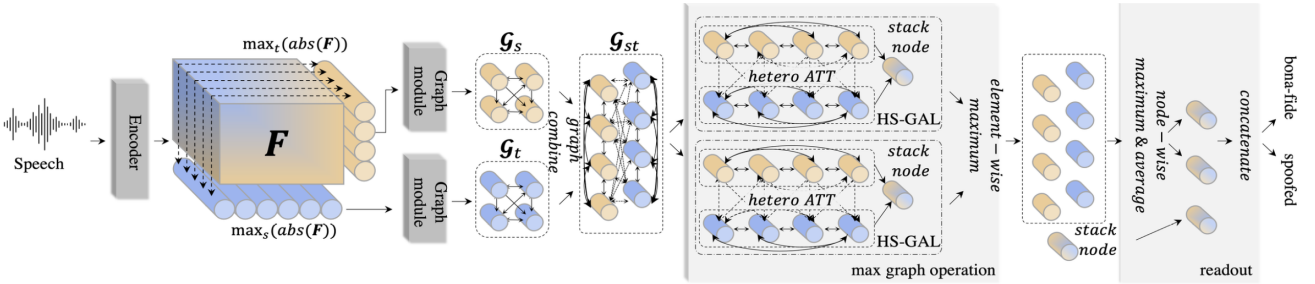


FIGURE 2 – Architecture AASIST telle que présentée dans le papier original (Jung et al., ICASSP 2022). De gauche à droite : encodeur (SincConv + blocs résiduels), extraction des vues spectrale $\mathcal{G}_s$ et temporelle $\mathcal{G}_t$, fusion hétérogène HtrgGAT avec nœud maître (*stack node*), et agrégation finale (*readout*).

Bloc 1 : *SincConv* ($bs, 64600$) $\rightarrow$ ($bs, 70, 64473$)

La première couche de convolution ne ressemble pas à une convolution standard. Ses 70 filtres sont contraints à être des passe-bandes sinusoidaux : chaque filtre est entièrement défini par deux paramètres appris, une fréquence basse f_{low} et une fréquence haute f_{high} , au lieu d’apprendre librement l’ensemble de ses coefficients. La forme du filtre est imposée par la physique : c’est un filtre rectangulaire dans le domaine fréquentiel, soit un sinus cardinal dans le domaine temporel, et seules ses bornes sont ajustées par rétropropagation. Cela réduit considérablement le nombre de paramètres par filtre et force une représentation interprétable : en sortie, les 70 canaux correspondent à 70 bandes fréquentielles réparties sur l’échelle mel (0–8 000 Hz). Un post-traitement $abs + \text{MaxPool2D}(3 \times 3) + \text{BN} + \text{SELU}$ suit immédiatement : abs élimine le signe pour ne garder que l’intensité, le pooling compresse la représentation d’un facteur $\times 9$, et BN+SELU normalisent et introduisent la non-linéarité. On obtient en sortie ($bs, 1, 23, 21491$) : un spectrogramme mel grossier, une image fréquence $\times$ temps.

Bloc 2 : *Encodeur résiduel* ($bs, 1, 23, 21491$) $\rightarrow$ ($bs, 64, 23, 29$)

Six blocs résiduels 2D s’enchaînent, chacun composé de deux $\text{Conv2D}(2 \times 3)$, d’une connexion résiduelle (*skip connection*) et d’un $\text{MaxPool}(1 \times 3)$ qui divise l’axe temporel par 3. Les canaux augmentent progressivement ($1 \rightarrow 32 \rightarrow 64$) pour capturer des patterns de plus en plus abstraits, tandis que le temps s’effondre de 21 491 à 29. La fréquence

reste fixe à 23, sans pooling sur cet axe, pour conserver toute l’information spectrale.

Bloc 3 : Séparation en deux vues (bs, 64, 23, 29) → vue S + vue T

Le tenseur (bs, 64, 23, 29) encode simultanément l’information fréquentielle (axe 23) et temporelle (axe 29). AASIST les sépare en deux résumés complémentaires par des opérations de max :

- **Vue spectrale** (bs, 23, 64) : max sur l’axe temporel; pour chaque bande fréquentielle, on retient sa valeur maximale sur toute la durée.
- **Vue temporelle** (bs, 29, 64) : max sur l’axe fréquentiel; pour chaque instant, on retient la valeur maximale sur toutes les bandes.

Ces deux vues sont ensuite traitées par deux graphes d’attention parallèles.

Bloc 4 : GAT spectral et temporel vue S → (bs, 11, 64) / vue T → (bs, 20, 64)

Un graphe d’attention (GAT, *Graph Attention Network*) permet de modéliser des relations non locales entre éléments d’une séquence. Dans la vue spectrale, chaque nœud du graphe représente une bande fréquentielle (23 nœuds); dans la vue temporelle, chaque nœud représente un instant (29 nœuds). Les arêtes entre nœuds ne sont pas fixes : leurs poids sont calculés dynamiquement par un mécanisme d’attention. Pour chaque paire de nœuds (i, j) , un coefficient α_{ij} mesure à quel point le nœud j est pertinent pour mettre à jour la représentation du nœud i . Le nœud i agrège alors une somme pondérée de ses voisins :

$$h'_i = \sum_j \alpha_{ij} \cdot Wh_j$$

L’intérêt par rapport à une convolution est fondamental : une convolution ne voit que des voisins locaux (bandes adjacentes ou instants proches), alors qu’un GAT peut relier n’importe quelle paire de nœuds. Cela permet de capturer des relations longue portée dans le spectre, par exemple une cohérence entre deux bandes fréquentielles distantes qui covarient dans les voix authentiques mais pas dans les deepfakes.

Après le GAT, un *graph pooling* appris élimine les nœuds les moins informatifs en leur attribuant un score de saillance : la vue spectrale passe de 23 à 11 nœuds (ratio 0,5), la vue temporelle de 29 à 20 nœuds (ratio 0,7).

Bloc 5 : HtrgGAT, fusion hétérogène (bs, 11, 64) + (bs, 20, 64) → (bs, 5, 32) + (bs, 10, 32) + nœud maître

C’est le composant le plus original d’AASIST et ce qui le distingue principalement de RawGAT-ST, qui traitait les deux branches spectrales et temporelles de façon entièrement indépendante. HtrgGAT construit un graphe *hétérogène* : les nœuds spectraux et temporels sont réunis dans un même graphe, avec des types d’arêtes différents selon la nature de la relation (spectrale-spectrale, temporelle-temporelle, ou spectrale-temporelle). Cela permet à un nœud de bande fréquentielle d’influencer un nœud temporel, et vice versa, ce qui est impossible dans deux graphes homogènes séparés. L’hypothèse sous-jacente est que les deepfakes introduisent des incohérences entre les deux domaines que seule une modélisation croisée peut détecter.

L’élément clé est le **nœud maître** : deux nœuds apprenables m_1 et m_2 sont ajoutés au graphe, connectés à tous les autres nœuds. Leur rôle est d’agréger un résumé global de l’ensemble du signal et de le redistribuer comme signal de contexte à chaque nœud lors de chaque couche d’attention. Le mécanisme est appliqué deux fois en parallèle (deux têtes d’attention), et on conserve le maximum des deux chemins pour chaque nœud.

En sortie, on dispose de trois représentations distinctes : les nœuds spectraux raffinés (bs, 5, 32), les nœuds temporels raffinés (bs, 10, 32), et le vecteur du nœud maître (bs, 1, 32).

Bloc 6 : Agrégation et classification → (bs, 160) → (bs, 2)

Cinq descripteurs globaux sont extraits des trois représentations de sortie de HtrgGAT par des opérations de max et mean sur les nœuds, puis concaténés en un vecteur de 160 dimensions (5×32). Le max capture les anomalies ponctuelles les plus saillantes; la mean capture les tendances globales. Une couche linéaire **Linear**(160 → 2) produit deux scores : un score bonafide et un score spoof. C’est la différence entre ces deux scores qui sert de décision finale.

2.2 Architectures comparées : RawNet2, RawGAT-ST et AASIST-L

Trois architectures de la même famille sont reproduites et comparées à AASIST.

AASIST-L est une version allégée d’AASIST (~ 85 k paramètres contre ~ 297 k), qui conserve la même structure en 6 blocs mais avec une capacité réduite.

RawNet2 [4] est le précurseur direct : il partage la première couche SincConv d’AASIST, mais la remplace ensuite par des blocs résiduels 1D et une agrégation par GRU (*Gated Recurrent Unit*). Il n’y a aucun graphe : le signal est traité comme une séquence temporelle pure. C’est la baseline la plus simple du groupe, avec ~ 379 k paramètres.

RawGAT-ST [3] est l’étape intermédiaire entre RawNet2 et AASIST : il introduit deux graphes d’attention séparés, l’un sur l’axe spectral et l’autre sur l’axe temporel, mais les traite de façon *indépendante*. AASIST franchit le pas suivant en les fusionnant via HtrgGAT. RawGAT-ST compte ~ 462 k paramètres.

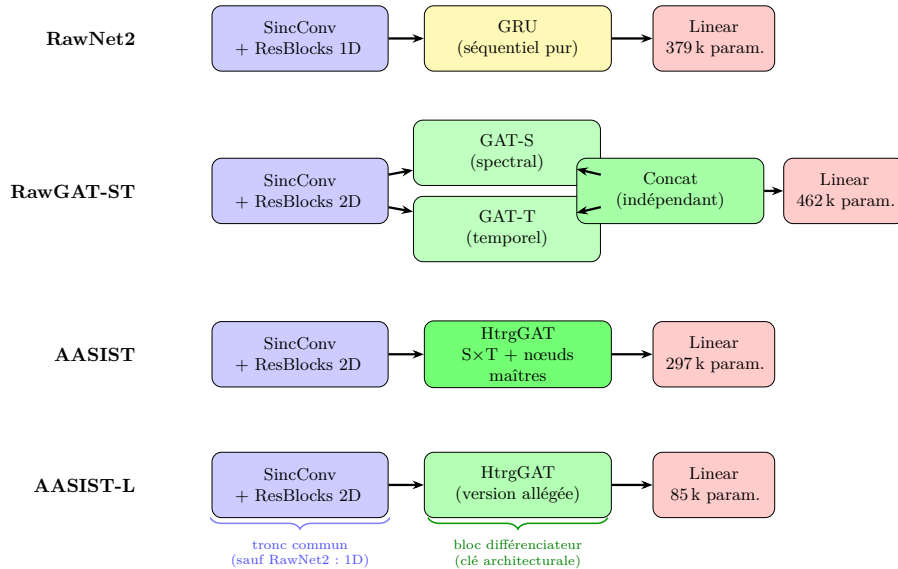


FIGURE 3 – Comparaison des quatre architectures. Le tronc commun (SincConv + blocs résiduels) est partagé. La différence principale réside dans le module central : RawNet2 utilise un GRU séquentiel sans graphe, RawGAT-ST deux GAT indépendants (S et T ne s’influencent pas), AASIST les fusionne via HtrgGAT avec nœuds maîtres, et AASIST-L reprend la même structure avec des dimensions réduites.

2.3 Variantes explorées

J’ai testé plusieurs variantes du protocole d’entraînement d’AASIST.

Variante tirée directement du code officiel :

- **Frequency augmentation** : en lisant le code des auteurs, j’ai constaté que le paramètre `Freq_aug` est implémenté dans `models/AASIST.py` mais désactivé par défaut. Le mécanisme masque aléatoirement une fenêtre de filtres SincConv (entre 0 et 20 filtres consécutifs) à chaque itération d’entraînement, de façon similaire au SpecAugment appliqué sur l’axe fréquentiel.

Variante motivée par la question de robustesse :

- **Noise augmentation** : l’évaluation sous bruit gaussien (section 4.6) mesure la robustesse d’un modèle entraîné sans bruit. Une question complémentaire est de savoir si entraîner avec du bruit modifie ce profil de robustesse. Les deux expériences sont donc conçues ensemble : l’une teste la robustesse à l’inférence, l’autre teste si l’entraînement sous bruit la transfère. Le bruit utilisé est un bruit gaussien additif (bruit blanc), à densité spectrale de puissance constante sur toutes les fréquences.
- **Fine-tuning** : cette variante n’était pas prévue initialement. C’est une réaction directe au résultat de noise augmentation (1,618 % EER, section 4.4) : introduire le bruit depuis zéro perturbe trop l’apprentissage, et partir du checkpoint officiel pour 30 epochs à $lr = 10^{-5}$ permet d’ajouter la robustesse au bruit sans repartir de zéro.

Une fusion de scores et une distillation de connaissance sont également testées :

Fusion pondérée. Les scores des quatre modèles sont normalisés (centrés-réduits) puis combinés avec des poids proportionnels à $1/\text{EER}_{eval}$ de chaque modèle, normalisés pour sommer à 1. La motivation est que des modèles d’architectures différentes font des erreurs sur des exemples différents, et leur combinaison devrait donc compenser

les faiblesses de chacun.

Distillation. AASIST (teacher, poids officiels gelés) est utilisé pour entraîner AASIST-L (student) avec une perte combinant cross-entropy sur les labels et divergence KL entre les distributions adoucies du student et du teacher :

$$\mathcal{L} = \alpha \cdot \text{CE}(\text{student}, y) + (1 - \alpha) \cdot \text{KL}(\text{student}_T \parallel \text{teacher}_T)$$

avec température $T = 4$ et $\alpha = 0,5$. L'idée est qu'AASIST produit des probabilités de sortie plus informatives que les simples labels 0/1, et qu'AASIST-L peut en bénéficier pendant l'entraînement.

3 Protocole expérimental

3.1 Dataset

Le dataset ASVspoof 2019 Logical Access (LA) contient des enregistrements de parole produits par 17 systèmes de synthèse ou de conversion vocale différents, chacun désigné par un code A01 à A19.

Le dataset est divisé en trois partitions :

- **Train** : utilisé pour entraîner le modèle.
- **Dev** : utilisé pendant l'entraînement pour choisir le meilleur checkpoint, sans jamais influencer directement les poids du modèle.
- **Eval** : le test final, jamais vu ni par l'entraînement ni par la sélection de checkpoint ; c'est sur cette partition que tous les résultats du rapport sont calculés.

Pour mesurer la capacité du modèle à généraliser à des attaques qu'il n'a jamais vues, les systèmes d'attaque utilisés en train/dev (A01 à A06) sont entièrement différents de ceux utilisés en eval (A07 à A19).

TABLE 1 – Répartition du dataset ASVspoof 2019 LA

Partition	Bona fide (voix réelle)	Spoof (voix falsifiée)
Train	2 580	22 800
Dev	2 548	22 296
Eval	7 355	63 882

Le dataset est fortement déséquilibré : environ 90 % des fichiers sont spoof, 10 % bona fide. Ce déséquilibre est inhérent à la construction du dataset : les enregistrements bona fide proviennent de vrais locuteurs et sont coûteux à collecter, tandis que les données spoof sont générées synthétiquement en grande quantité. Ce déséquilibre est compensé dans l'entraînement par une cross-entropy pondérée (poids 0,9 pour bona fide, 0,1 pour spoof), pour éviter que le modèle apprenne simplement à tout classer comme spoof.

3.2 Métriques d'évaluation

EER (*Equal Error Rate*) : le modèle produit un score continu pour chaque audio. Pour prendre une décision, on fixe un seuil : au-dessus, c'est spoof ; en dessous, c'est bona fide. Ce seuil crée toujours deux types d'erreurs : les faux rejets (une vraie voix classée spoof) et les fausses acceptations (un deepfake classé bona fide). L'EER est le taux d'erreur au seuil où ces deux erreurs sont exactement égales. Un EER de 0 % signifie une séparation parfaite, 50 % correspond à une décision aléatoire.

min-tDCF (*minimum tandem Detection Cost Function*) : c'est la métrique officielle du défi ASVspoof 2019. Elle évalue le CM non pas seul, mais en tandem avec un système ASV, ce qui est plus proche d'un déploiement réel. La formule est :

$$\text{t-DCF}(s) = C_1 \cdot P_{\text{miss}, CM}(s) + C_2 \cdot P_{\text{fa}, CM}(s)$$

où :

- s est le seuil de décision du CM ;
- $P_{\text{miss}, CM}(s)$: proportion de deepfakes classés à tort comme bona fide ;
- $P_{\text{fa}, CM}(s)$: proportion de voix réelles classées à tort comme spoof ;
- C_1 et C_2 : coefficients de coût qui tiennent compte de $P_{\text{spoof}} = 0,05$ (les attaques sont rares dans le trafic réel) et des performances du système ASV associé.

Le min-tDCF est le minimum de cette fonction sur tous les seuils possibles. Un score sous 0,05 est considéré

compétitif. Deux modèles peuvent avoir le même EER mais un min-tDCF différent si leurs erreurs sont réparties différemment entre les deux types, c’est pourquoi le min-tDCF est plus discriminant que l’EER seul.

3.3 Détails d’implémentation

Voici les hyperparamètres utilisés pour entraîner tous les modèles. Ils sont fournis par le papier original et les configs officielles d’AASIST.

TABLE 2 – Hyperparamètres d’entraînement

Hyperparamètre	Valeur	Rôle
Optimiseur	Adam	Ajuste les poids selon le gradient
Learning rate	10^{-4}	Taille du pas à chaque mise à jour
β_1, β_2	0,9 / 0,999	Mémoire des gradients passés
Weight decay	10^{-4}	Pénalité sur les poids trop grands
Scheduler	cosine annealing	Diminue le LR selon une courbe en cosinus
SWA	activé en fin d’entraînement	Moyenne les derniers checkpoints
Cross-entropy pondérée	0,1 / 0,9 (bona fide / spoof)	Compense le déséquilibre des classes
Batch size	24 (32 pour RawNet2)	Nombre d’exemples par mise à jour
Epochs	100	Passages complets sur le jeu d’entraînement
Longueur d’entrée	4 s (64 600 ech. à 16 kHz)	Découpage aléatoire à l’entraînement

Les jobs ont été soumis sur le cluster GPU de Télécom Paris.

3.4 Remarques d’implémentation

Le code officiel a été publié en 2021 et deux ajustements de compatibilité ont été nécessaires :

- **evaluation.py : np.float supprimé dans NumPy 1.24+.** Le code utilisait `np.float`, alias de `float` supprimé depuis NumPy 1.24. Remplacé par `float`.
- **Configs RawGATST_baseline.conf et RawNet2_baseline.conf : chemin de poids inaccessible.** Les deux configs pointaient vers des poids hébergés sur le serveur interne de NAVER, inaccessible depuis l’extérieur. Redirigés vers les checkpoints téléchargés localement.

4 Résultats et commentaires

4.1 Validation des poids officiels

TABLE 3 – Validation des poids officiels sur ASVspoof 2019 LA eval

Modèle	EER (%)	min-tDCF	EER papier	tDCF papier
AASIST	0,8295	0,02753	0,83	0,0275
AASIST-L	0,9917	0,03090	0,99	0,0309

Les résultats correspondent au papier à la quatrième décimale près, ce qui valide la mise en place expérimentale (dataset, protocole, code d’évaluation) avant de passer à l’entraînement from scratch.

4.2 Reproduction de l’entraînement pour les modèles de référence

TABLE 4 – Entraînements from scratch

Modèle	EER (%)	min-tDCF	Checkpoint	Notes
AASIST (best eval, ep60) [‡]	1,335	0,03871	60 (eval)	référence pour les ablations
AASIST (best dev, ep98) [‡]	1,480	0,04951	98 (dev)	protocole standard ; utilisé pour le breakdown
AASIST-L	0,9917	0,0309	76 (dev)	Égale exactement le papier
RawGAT-ST	1,67	0,0546	85 (dev)	SWA : 1,672 % / 0,0551
RawNet2	4,231	0,113	88 (dev)	Pic eval ep54 (4,23 %), dégradation ensuite

[‡]AASIST présente une divergence dev/eval : l’EER eval atteint son minimum à ep60 (1,335 %) puis remonte, tandis que l’EER dev continue de baisser jusqu’à ep98 (best dev). Le checkpoint ep98 est utilisé pour le breakdown (section 4.3) conformément au protocole standard ; ep60 sert de référence pour les ablations (section 4.4) car il représente le meilleur résultat from scratch atteint.

AASIST et AASIST-L reproduisent les résultats publiés ; RawGAT-ST et RawNet2 approchent les valeurs de la littérature sans qu’une comparaison directe ne soit étayée dans ce rapport. AASIST-L[†] égale le papier à la quatrième décimale : les deux métriques correspondent exactement aux valeurs publiées.

AASIST from scratch atteint 1,335 % EER (best epoch 98), soit 0,5 point au-dessus des poids officiels (0,83 %).

RawNet2 illustre un cas classique de sur-ajustement au dev set : entre l’epoch 54 et l’epoch 88, l’EER dev continue de baisser (0,708 % → 0,551 %) alors que l’EER eval se dégrade (4,23 % → 5,11 % avant de revenir à 4,46 %). Le SWA final retombe au niveau de l’epoch 54 (4,231 %), sans vraiment corriger le sur-ajustement, juste en le lissant.

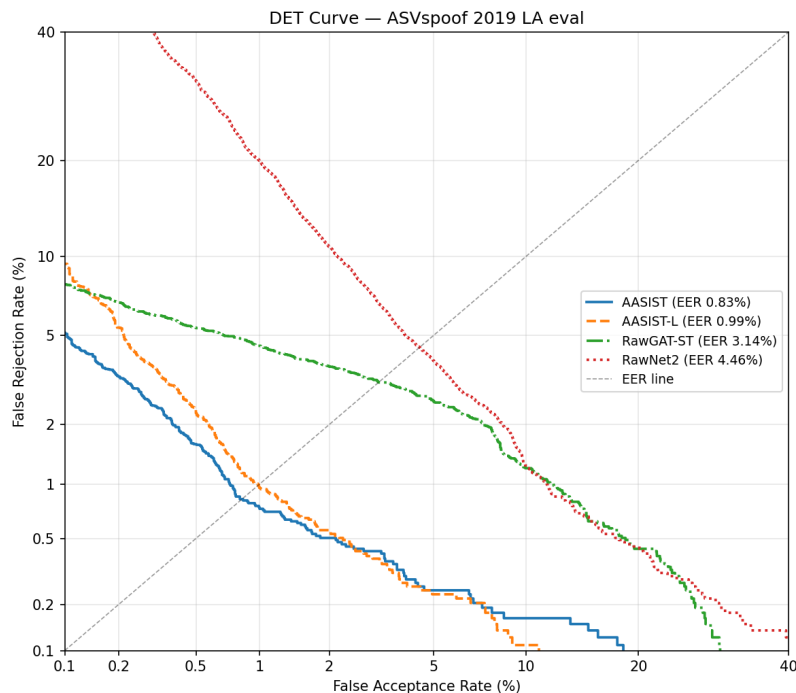


FIGURE 4 – Courbe DET des 4 modèles (sans ensemble). AASIST domine sur toute la plage de seuils ; RawNet2 reste nettement au-dessus.

4.3 Analyse par type d’attaque

Plutôt que de calculer un EER global sur toutes les attaques à la fois, on isole chaque type d’attaque A_k et on mesure à quel point le modèle arrive à le distinguer des vraies voix. Cela permet de voir sur quelles attaques précisément chaque modèle échoue.

TABLE 5 – EER par attaque pour les 4 modèles, tous entraînés from scratch. Les valeurs AASIST correspondent au checkpoint best dev ep98 (EER global 1,480 %, protocole standard).

Attaque	AASIST	AASIST-L	RawGAT-ST	RawNet2
A09	0,08 %	0,00 %	0,02 %	0,14 %
A13	0,33 %	0,20 %	0,30 %	0,24 %
A14	0,31 %	0,06 %	0,59 %	0,63 %
A11	0,75 %	0,14 %	0,55 %	1,08 %
A08	0,53 %	0,20 %	0,42 %	3,13 %
A07	1,49 %	0,55 %	1,53 %	2,76 %
A16	0,80 %	0,63 %	1,26 %	0,86 %
A19	0,68 %	0,71 %	0,94 %	1,75 %
A12	1,51 %	0,69 %	1,67 %	2,77 %
A10	1,73 %	1,00 %	1,96 %	2,67 %
A15	1,10 %	0,57 %	1,10 %	2,50 %
A17	1,48 %	1,92 %	2,18 %	8,08 %
A18	3,37 %	3,05 %	4,70 %	14,26 %
Moyenne	1,09 %	0,75 %	1,32 %	3,14 %

Moyenne = moyenne non pondérée des EER par attaque, $\neq$ EER global calculé sur l'ensemble du corpus.

Observations. A09 et A13 sont triviales pour tous les modèles : ce sont des systèmes TTS plus anciens dont les artefacts sont facilement détectables. À l'autre extrême, A17 et A18 correspondent à des vocoders neuronaux récents et concentrent l'essentiel des erreurs pour tous les modèles.

RawNet2 est de loin le plus en difficulté sur A18 (14,26 %) et n'est premier sur aucune attaque.

RawGAT-ST est globalement bon mais souffre également d'A18 (4,70 %).

AASIST-L domine sur 11 des 13 attaques et présente le meilleur profil from scratch : son EER moyen par attaque (0,75 %) est le plus bas des quatre, avec une bonne homogénéité.

AASIST from scratch reste compétitif sur A09 et A17, mais est plus irrégulier qu'AASIST-L : il est meilleur sur A07 et A17 mais nettement pire sur A11, A12 et A10. Cela illustre que les poids obtenus from scratch n'ont pas convergé aussi bien que les poids officiels, et que l'hétérogénéité des erreurs entre les deux modèles reste exploitable par une fusion.

4.4 Variantes du protocole d'entraînement

Les variantes from scratch sont comparées à AASIST from scratch ep60 (1,335 %, meilleur EER eval atteint), et non aux poids officiels (0,83 %) : comparer à l'officiel mêlerait l'effet de la modification et le gap naturel entre from scratch et officiel. Le fine-tuning, qui part des poids officiels, est comparé séparément à ceux-ci.

TABLE 6 – Variantes d'entraînement : chaque modèle comparé à sa propre référence from scratch

Variante	EER (%)	min-tDCF	Δ EER	Verdict
<i>AASIST — référence : 1,335 %</i>				
AASIST from scratch (référence)	1,335	0,03871	—	—
AASIST + frequency augmentation	1,142	0,03646	−0,19 pt	légère amélioration
AASIST + noise augmentation	1,618	0,05401	+0,28 pt	dégradation
AASIST fine-tuning + noise_aug*	1,224	0,0383	+0,39 pt	dégradation
<i>AASIST-L — référence : 0,9917 %</i>				
AASIST-L from scratch (référence)	0,9917	0,03090	—	—
AASIST-L + frequency augmentation	1,102	0,03583	+0,11 pt	légère dégradation globale

* *Fine-tuning : part des poids officiels (0,83 %), Δ calculé par rapport à ce point de départ.*

Observations. La frequency augmentation améliore légèrement l'EER global (−0,19 pt). La noise augmentation

est la seule variante clairement dégradante (+0,28 pt) : le dataset d'évaluation est propre, et le bruit d'entraînement brouille les caractéristiques spectrales qu'AASIST utilise pour détecter les deepfakes. Label smoothing (1,360 %, min-tDCF 0,04516) et scheduler adaptatif (1,425 %, min-tDCF 0,04647) ont également été testés. En EER, les écarts sont faibles (+0,025 pt et +0,09 pt) ; en min-tDCF en revanche, la dégradation est notable (respectivement +17 % et +20 % par rapport à la référence 0,03871), ce qui relativise le caractère neutre de ces variantes si l'on regarde le min-tDCF.

Le fine-tuning depuis les poids officiels (+0,39 pt par rapport à 0,83 %) est pire que le point de départ, mais reste meilleur que la noise augmentation from scratch (1,618 %) : partir d'une représentation déjà bien apprise limite la dégradation.

Sur l'attaque la plus difficile, A18, la frequency augmentation apporte le gain le plus net :

TABLE 7 – EER sur l'attaque A18 pour chaque variante

	from scratch	freq_aug	noise_aug	fine-tuning
AASIST	3,37 %	2,99 %	4,70 %	n.d.
AASIST-L	3,05 %	2,64 %	—	—

Observations. Sur l'attaque A18, la frequency augmentation améliore les deux modèles : AASIST passe de 3,37 % à 2,99 % (tout en gagnant également en EER global), et AASIST-L de 3,05 % à 2,64 % (malgré une légère dégradation globale de +0,11 pt). A17 et A18 correspondent aux vocoders neuronaux de dernière génération, précisément les attaques les plus représentatives des menaces actuelles. La frequency augmentation améliore donc les attaques les plus dures sans trop dégrader les autres, ce qui suggère un meilleur équilibre sur des datasets plus récents (ASVspoof 5, In-the-Wild) où ce type de générateurs domine. Ce résultat reste à confirmer sur plusieurs seeds.

Tables complètes par attaque pour chaque variante : annexe A.

4.5 Fusion de scores

Les profils d'erreur complémentaires observés en section 4.3 motivent une fusion des scores des quatre modèles, normalisés (centrés-réduits) puis combinés avec des poids proportionnels à $1/\text{EER}_{eval}$. L'ensemble a été calculé avec les scores des poids officiels AASIST (0,83 %), les scores from scratch n'étant pas encore disponibles à ce moment. Les trois autres modèles utilisent leurs scores from scratch.

TABLE 8 – Poids de la fusion pondérée (AASIST : poids officiels ; autres : from scratch)

Modèle	EER utilisé	Poids
AASIST	0,83 % (officiel)	0,395
AASIST-L	0,99 %	0,331
RawGAT-ST	1,67 %	0,196
RawNet2	4,23 %	0,078

Observations. La fusion pondérée bat tous les modèles individuels, y compris AASIST officiel (0,83 %) : 0,73 % d'EER et -21 % de tDCF. Le gain est concentré sur les attaques difficiles (A17 : 1,48 % $\rightarrow$ 1,18 %, meilleur from scratch individuel étant AASIST ; A18 : 3,05 % $\rightarrow$ 2,46 %), ce qui confirme que les quatre modèles se trompent sur des exemples au moins partiellement différents. Des modèles d'architectures différentes font des erreurs sur des exemples différents : c'est ça qui rend la fusion utile, plus que le niveau de chaque modèle seul.

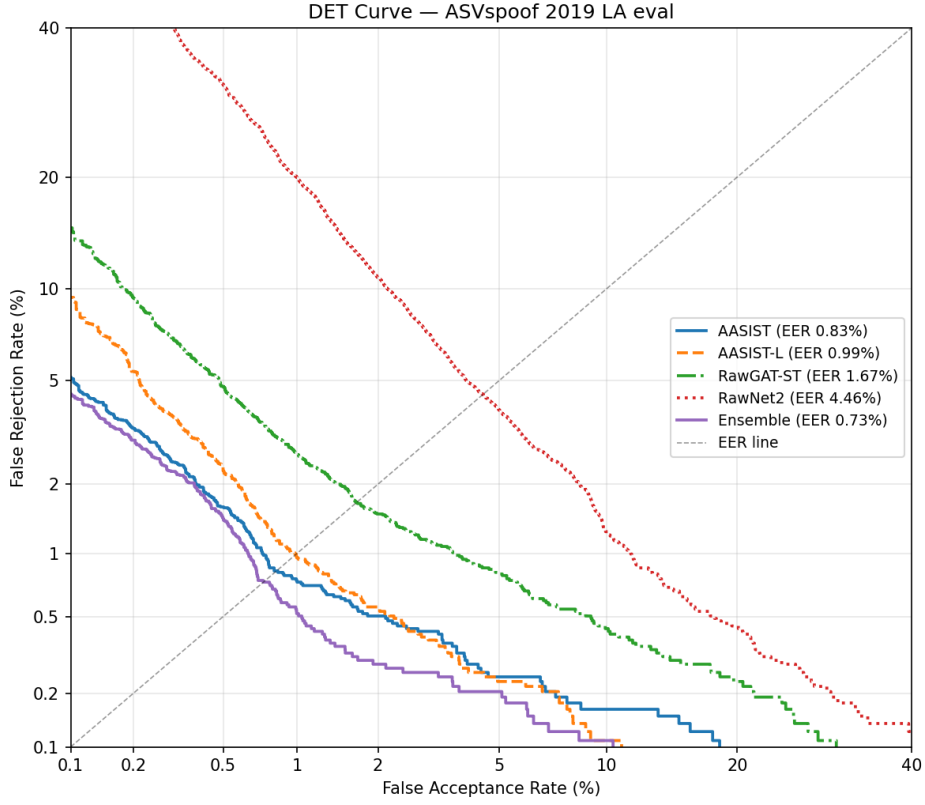


FIGURE 5 – Courbe DET avec la fusion pondérée. La fusion (trait épais) passe sous tous les modèles individuels, notamment sur les attaques difficiles.

4.6 Robustesse au bruit

Le protocole consiste à ajouter du bruit gaussien blanc à l’audio d’évaluation à différents niveaux de SNR (30, 20, 10, 5 et 0 dB) avec une graine fixée (1234) pour la reproductibilité. Le modèle évalué est AASIST avec ses poids officiels.

TABLE 9 – EER en fonction du niveau de bruit

SNR (dB)	EER (%)
Propre (∞)	0,83
30	1,02
20	4,20
10	9,36
5	25,90
0	63,10

Observations. La dégradation est faible à 30 dB (+0,2 point), puis s’accélère brutalement entre 20 dB et 10 dB ($\times 5$ puis $\times 11$ par rapport à la référence propre). À 0 dB, l’EER atteint 63,10 %, soit pire qu’un classifieur aléatoire (50 % correspondrait à une décision au hasard). Ce résultat s’explique directement par la composition du dataset : ASVspoof 2019 LA ne contient aucun enregistrement bruité, et AASIST travaille sur des caractéristiques spectrales fines (bandes SincConv, relations entre fréquences dans le GAT spectral) que le bruit additif efface en premier.

Ce résultat donne du recul sur le fine-tuning avec noise augmentation : malgré sa dégradation en EER global (1,22 % contre 0,83 %), il pourrait mieux tenir à des SNR intermédiaires. Cette comparaison n’a pas été faite ici.

4.7 Distillation AASIST $\rightarrow$ AASIST-L

L’hypothèse testée est qu’AASIST-L distillé depuis AASIST officiel dépasse AASIST-L entraîné from scratch (référence : 0,99 % EER). La perte combinée est $\mathcal{L} = 0,5 \cdot \text{CE}(\text{student}, y) + 0,5 \cdot \text{KL}(\text{student}_T \| \text{teacher}_T)$ avec $T = 4$, entraînée sur 100 epochs.

TABLE 10 – Distillation AASIST $\rightarrow$ AASIST-L

Modèle	EER (%)	min-tDCF
AASIST officiel (teacher)	0,8295	0,02753
AASIST-L from scratch (référence student)	0,9917	0,03090
AASIST-L distillé	1,390	0,03826

Observations. La distillation dégrade la performance par rapport à l’entraînement from scratch d’AASIST-L (+0,40 point d’EER). Deux raisons expliquent probablement cet échec : AASIST produit des scores très tranchés (peu d’information « molle » à transférer à $T = 4$), et AASIST-L from scratch atteint déjà 0,99 %, soit seulement 0,16 point derrière le teacher — il n’y a pas grand chose à apprendre.

TABLE 11 – Breakdown par attaque : distillation vs from scratch

Attaque	AASIST-L distillé	AASIST-L from scratch
A09	0,017 %	0,00 %
A13	0,227 %	0,20 %
A14	0,081 %	0,06 %
A11	0,105 %	0,14 %
A08	0,791 %	0,20 %
A07	0,424 %	0,55 %
A15	0,390 %	0,57 %
A16	1,100 %	0,63 %
A19	1,362 %	0,71 %
A12	0,587 %	0,69 %
A10	0,611 %	1,00 %
A17	2,136 %	1,92 %
A18	3,562 %	3,05 %

5 Limitations

Un seul run par configuration. Aucune des variantes (section 4.4) n’a été répétée sur plusieurs seeds. Les écarts observés (freq_aug −0,19 pt, noise_aug +0,28 pt) ne sont pas garantis statistiquement significatifs, mais +0,28 pt est suffisamment grand pour être probablement réel.

Bruit gaussien blanc uniquement. Le bruit gaussien est le modèle de dégradation le plus simple ; il ne représente pas des conditions réalistes comme le bruit de foule, la réverbération ou la compression par codec.

Pas d’évaluation cross-dataset. Les quatre modèles sont entraînés et évalués sur ASVspoof 2019 LA exclusivement. Leur généralisation à des datasets plus récents (ASVspoof 5, In-the-Wild) n’est pas mesurée, alors que les attaques A07–A19 ont été générées en 2019 et ne reflètent pas les générateurs vocaux actuels (2024–2025).

Pas d’attaque adaptative. Un adversaire qui connaît l’architecture AASIST pourrait construire des exemples spécifiquement conçus pour la tromper. Ce type d’attaque n’a pas été testé ici.

6 Conclusion

Bilan technique. Ce projet valide l’implémentation officielle d’AASIST et reproduit l’entraînement des quatre architectures de référence sur ASVspoof 2019 LA. L’analyse attaque par attaque a permis de comprendre quels modèles échouent sur quels types de deepfakes et pourquoi (notamment la vulnérabilité commune sur A17/A18, les vocoders neuronaux), et d’exploiter ces profils complémentaires dans une fusion pondérée qui dépasse tous les modèles individuels (0,73 % EER). L’étude des variantes, comparées à AASIST from scratch, montre que la plupart des modifications changent peu : seule la noise augmentation dégrade clairement, et la frequency augmentation améliore un peu les attaques les plus difficiles.

Retour personnel. Ce projet a été l’occasion de mener mon premier projet d’IA seul. C’est la première fois que j’utilisais un cluster de calcul : soumission de jobs SLURM, gestion des ressources GPU, débogage à distance. C’est aussi la première fois que j’entraînais des modèles de cette taille depuis zéro.

Ce projet m'a permis d'approfondir de manière appliquée des notions vues en cours de deep learning au P3 en les confrontant à des choix concrets. J'ai particulièrement apprécié le travail de comparaison des modèles et d'analyse afin de chercher à comprendre *pourquoi* un modèle est meilleur sur telle attaque et pas une autre.

Ce travail était d'autant plus intéressant qu'il est très différent de ce que je fais en entreprise chez Argon, où mon travail est davantage orienté analyse de données et SQL. Ce projet m'a donné envie de demander à travailler sur des missions plus data science / machine learning.

Références

- [1] Jung, J.-w., Heo, H., Tak, H., Shim, H., Chung, J. S., Lee, B.-J., Yu, H.-J., & Evans, N. (2022). *AASIST : Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks*. ICASSP 2022.
- [2] Kinnunen, T. et al. (2020). *ASVspoof 2019 : A large-scale public database of synthetic, converted and replayed speech*. Computer Speech & Language.
- [3] Jung, J.-w. et al. (2021). *RawGAT-ST : Raw waveform graph attention network for speech anti-spoofing*. Inter-speech 2021.
- [4] Tak, H., Patino, J., Todisco, M., Nautsch, A., Evans, N., & Larcher, A. (2021). *End-to-end anti-spoofing with RawNet2*. ICASSP 2021.
- [5] Kinnunen, T. et al. (2018). *t-DCF : A Detection Cost Function for the Tandem Assessment of Spoofing Countermeasures and Automatic Speaker Verification*. Odyssey 2018.
- [6] Müller, R., Kornblith, S., & Hinton, G. (2019). *When Does Label Smoothing Help ?* NeurIPS 2019.
- [7] Wong, W. (2020). *What is Label Smoothing ?* Towards Data Science. <https://towardsdatascience.com/what-is-label-smoothing-108debd7ef06>

A Breakdown complet par attaque pour les variantes

TABLE 12 – EER par attaque : frequency augmentation et noise augmentation

Attaque	noise_aug	AASIST+freq_aug	AASIST-L+freq_aug
A09	0,261	0,00	0,04
A13	0,587	0,15	0,15
A14	0,489	0,29	0,34
A11	0,954	0,34	0,30
A08	1,083	0,39	0,31
A07	1,002	1,06	0,96
A15	1,222	0,83	1,04
A16	0,913	0,73	0,63
A19	0,937	0,61	0,65
A12	1,321	0,75	0,94
A10	1,752	1,24	1,10
A17	1,623	1,81	1,91
A18	4,703	2,99	2,64

Toutes valeurs en %. Référence AASIST-L no-aug sur A18 : 3,05 %.

B Historique d'entraînement RawNet2 (best checkpoints)

TABLE 13 – Évolution des EER dev et eval pour RawNet2

Epoch (best dev)	dev EER (%)	eval EER (%)	Commentaire
28	0,900	5,83	premier best
40	0,825	5,06	
44	0,825	4,69	
46	0,785	4,62	
54	0,708	4,23	meilleur eval
70	0,708	5,11	dev stable, eval se dégrade
73	0,628	4,53	
82	0,551	4,65	
88	0,551	4,46	best dev, pire eval
Final SWA	—	4,231	≈ niveau ep54

Entre l'époch 54 et 88, le dev EER continue de baisser tandis que l'eval EER se dégrade : signe net de sur-ajustement au dev set. Le SWA final retombe au niveau de l'époch 54.

C Breakdown EER par attaque

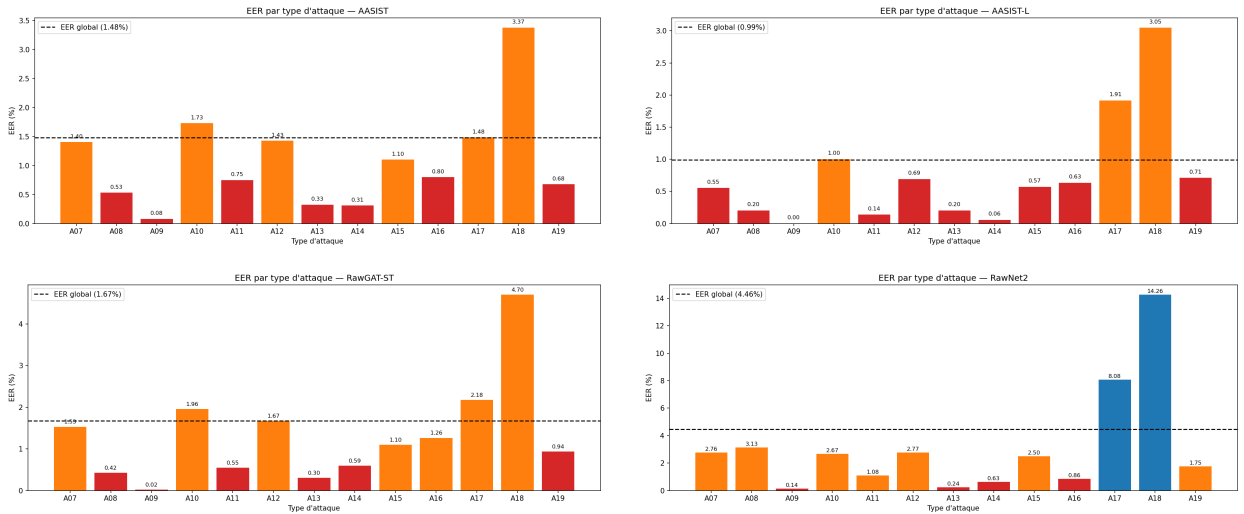


FIGURE 6 – EER par attaque pour les 4 modèles (haut gauche : AASIST, haut droite : AASIST-L, bas gauche : RawGAT-ST, bas droite : RawNet2).